

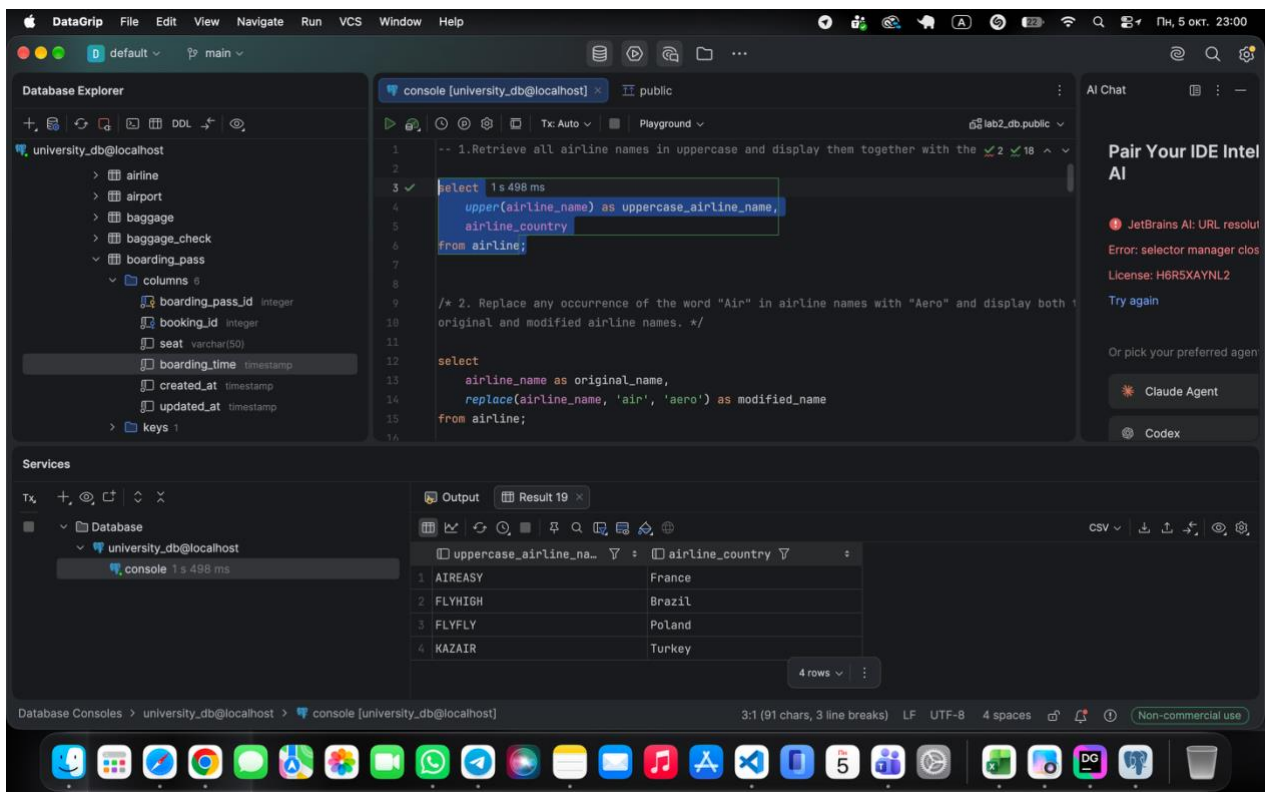
Course name: Databases

Laboratory work 4

Submission format: please work with the database from the prev lab work and provide fullpage screenshots that covers the code and results of each task in the PDF format, also provide sql files.

Required tasks:

1. Retrieve all airline names in uppercase and display them together with their country



The screenshot shows the DataGrip IDE interface. On the left, the Database Explorer shows a connection to 'university_db@localhost' with a schema containing tables like 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', 'columns', 'keys', 'seats', and 'times'. The main console window shows a SQL query being executed:

```
-- 1. Retrieve all airline names in uppercase and display them together with their country
select
  upper(airline_name) as uppercase_airline_name,
  airline_country
from airline;
```

 The execution time is 1s 498 ms. Below the console, the 'Result 19' window displays the query results in a table format:

uppercase_airline_name	airline_country
AIREASY	France
FLYHIGH	Brazil
FLYFLY	Poland
KAZAIR	Turkey

 The bottom status bar indicates the file is 3:1 (91 chars, 3 line breaks) in UTF-8 encoding with 4 spaces.

2. Replace any occurrence of the word "Air" in airline names with "Aero" and display both the original and modified airline names.

The screenshot shows the DataGrip IDE interface. On the left, the Database Explorer shows a database named 'university_db@localhost' with tables 'airline', 'airport', 'baggage', 'baggage_check', and 'boarding_pass'. The 'airline' table is selected, showing its columns: 'boarding_pass_id' (integer), 'booking_id' (integer), 'seat' (varchar(50)), 'boarding_time' (timestamp), 'created_at' (timestamp), and 'updated_at' (timestamp). The main editor shows a SQL console with the following code:

```
/* 2. Replace any occurrence of the word "Air" in airline names with "Aero" and display both original and modified airline names. */  
  
select 387 ms  
    airline_name as original_name,  
    replace(airline_name, 'air', 'aero') as modified_name  
from airline;  
  
-- 3. Find all flight numbers that coordinates with both airline 1 and airline 2.  
  
select flight_id  
from flights  
where airline_id = 1  
intersect
```

The output window shows the result of the first query, displaying a table with two columns: 'original_name' and 'modified_name'. The results are:

original_name	modified_name
AirEasy	AirEasy
FlyHigh	FlyHigh
FlyFly	FlyFly
KazAir	KazAir

The status bar at the bottom indicates the file is 12:1 (113 chars, 3 line breaks) in UTF-8 encoding with 4 spaces.

3. Find all flight numbers that coordinates with both airline 1 and airline 2.

The screenshot shows the DataGrip IDE interface. The main editor shows a SQL console with the following code:

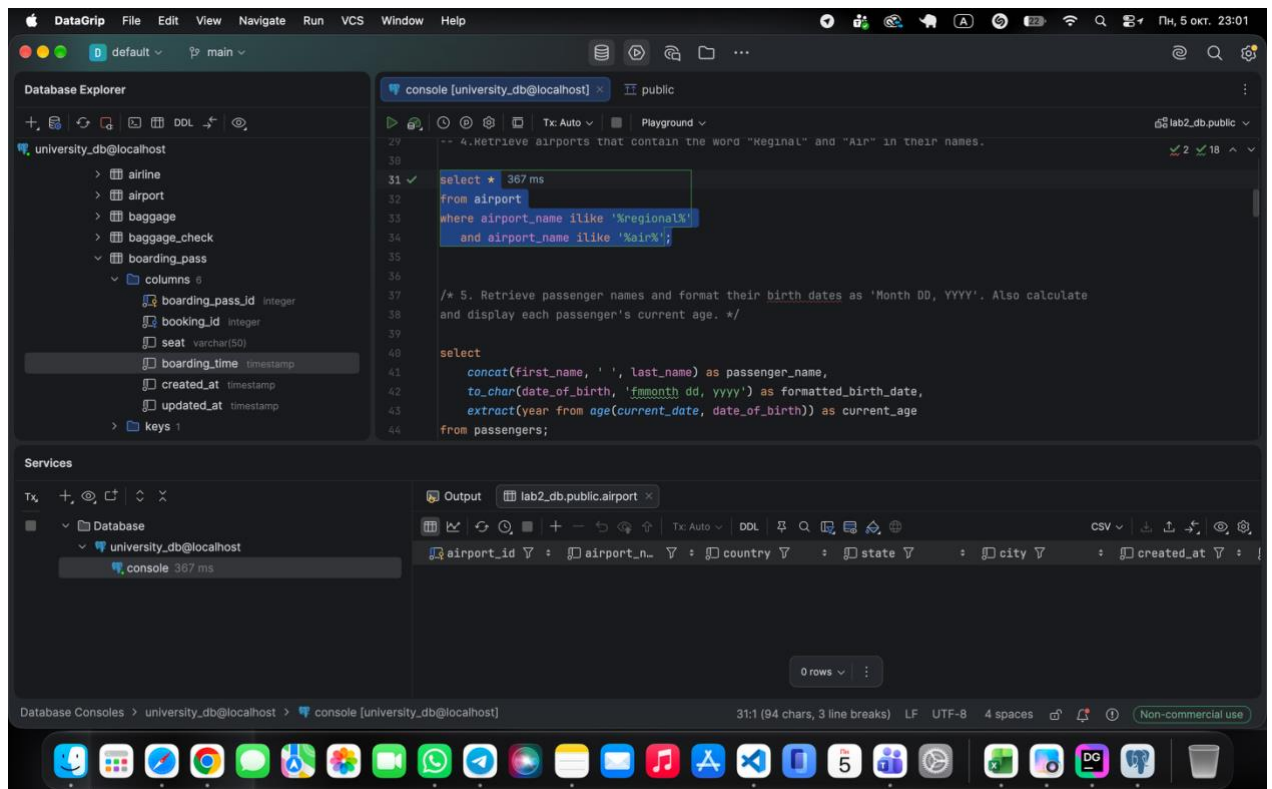
```
-- 3. Find all flight numbers that coordinates with both airline 1 and airline 2.  
  
select 366 ms  
    flight_id  
from flights  
where airline_id = 1  
intersect  
select flight_id  
from flights  
where airline_id = 2;  
  
-- 4. Retrieve airports that contain the word "Reginal" and "Air" in their names.  
  
select *  
from airport
```

The output window shows the result of the second query, displaying a table with one column: 'flight_id'. The results are empty, showing 0 rows.

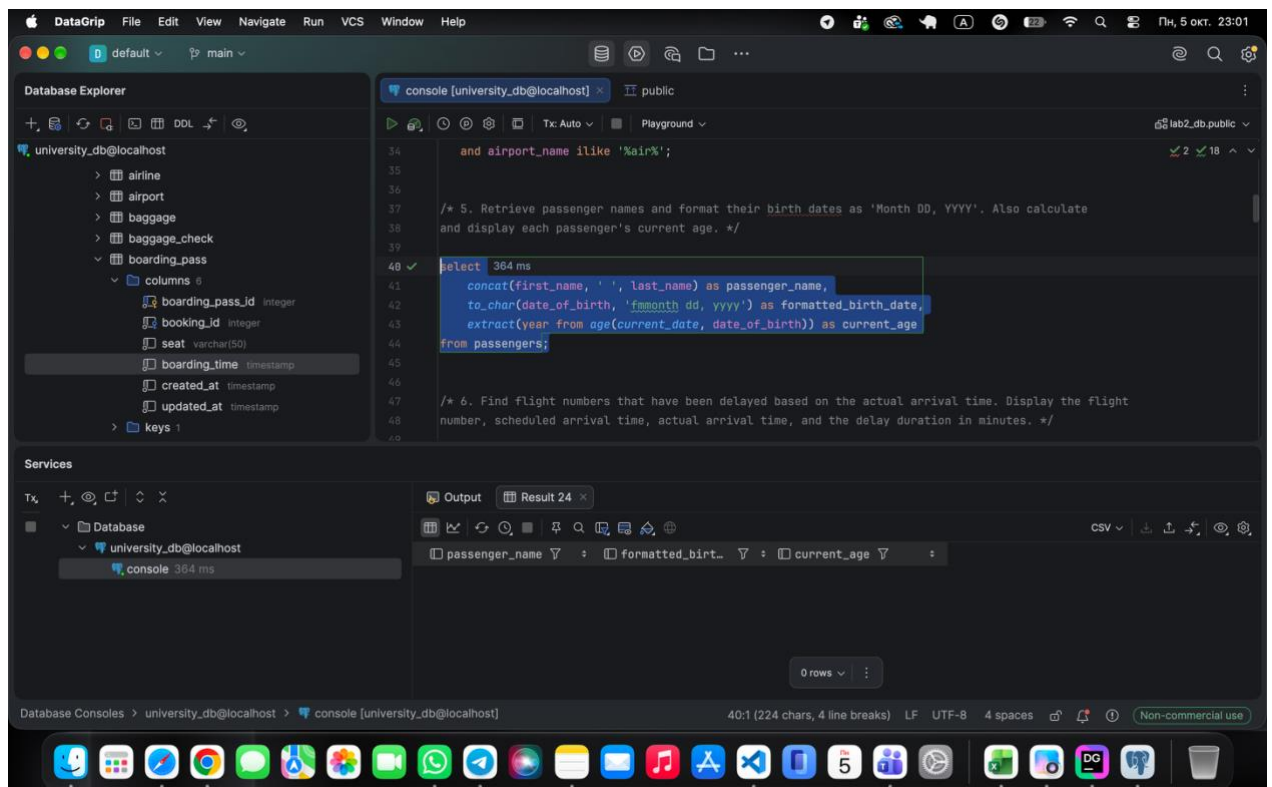
flight_id

The status bar at the bottom indicates the file is 20:1 (112 chars, 6 line breaks) in UTF-8 encoding with 4 spaces.

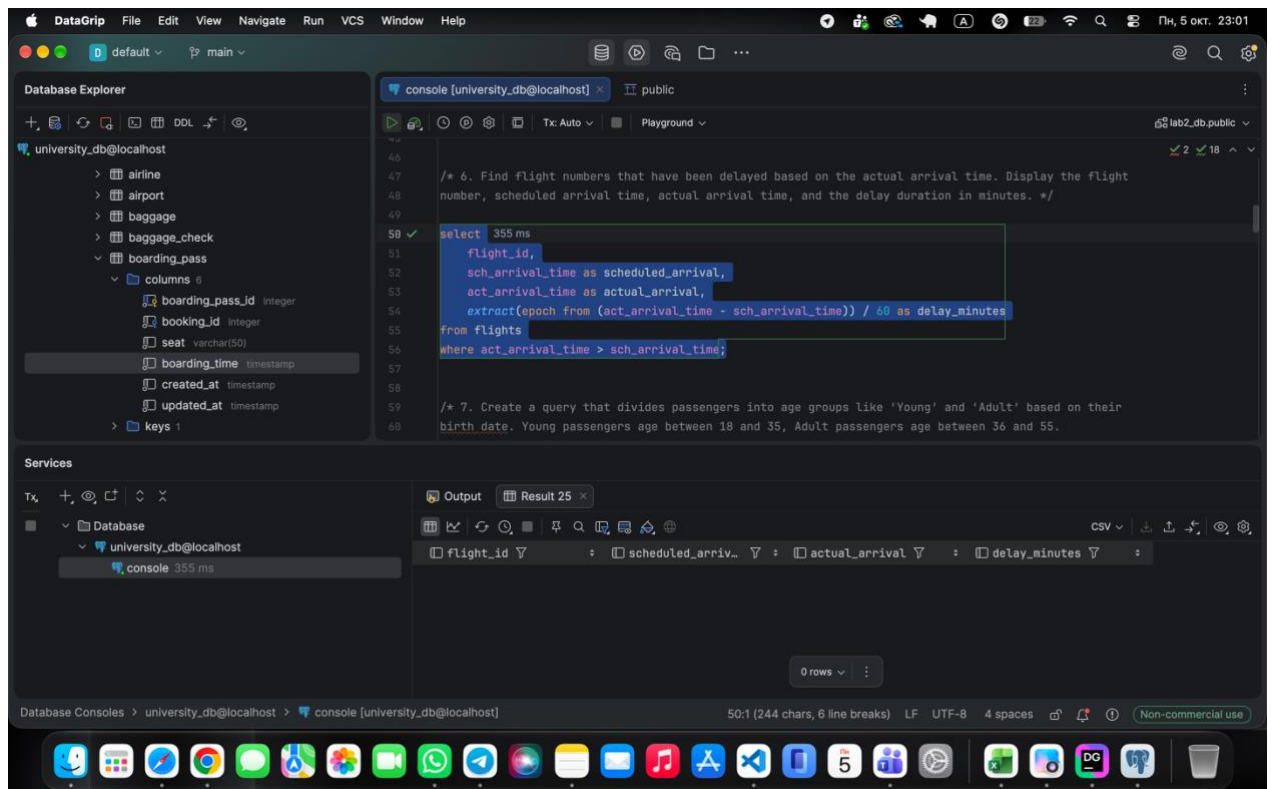
4. Retrieve airports that contain the word "Reginal" and "Air" in their names.



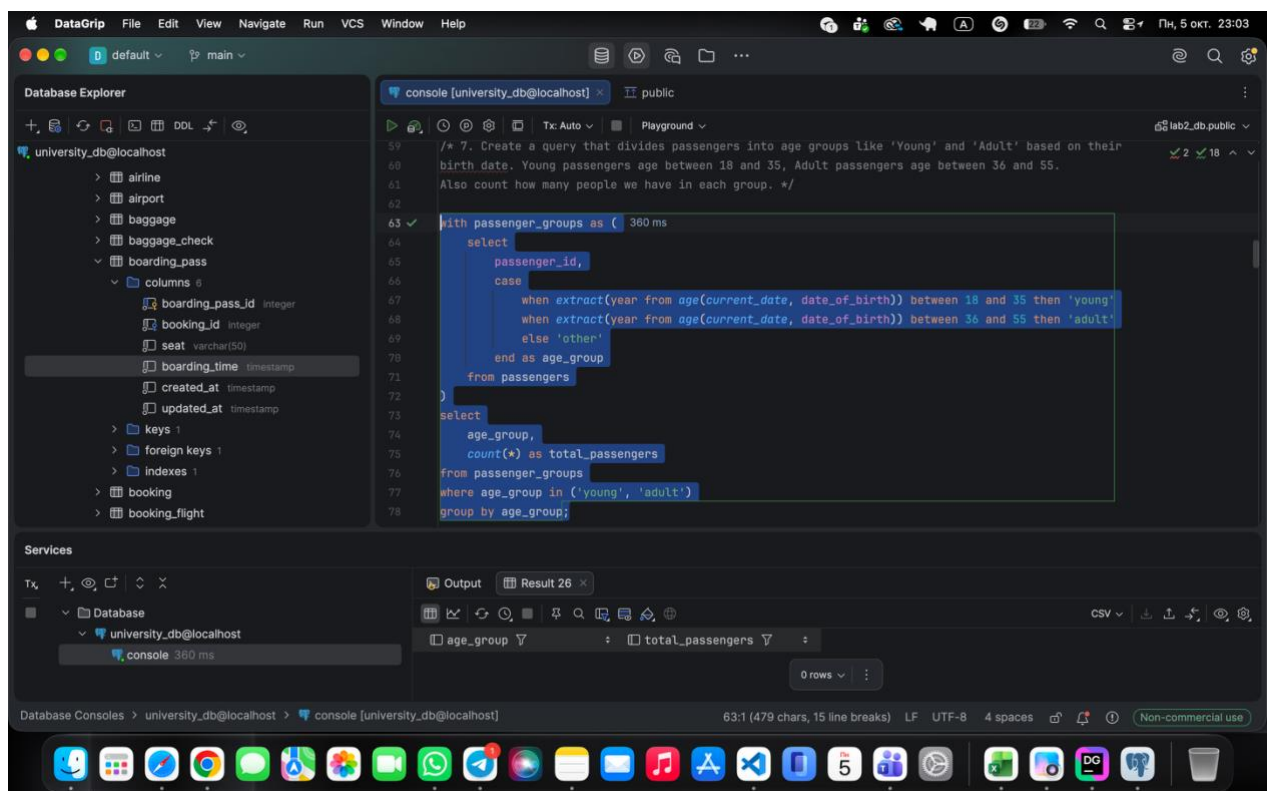
5. Retrieve passenger names and format their birth dates as 'Month DD, YYYY'. Also calculate and display each passenger's current age.



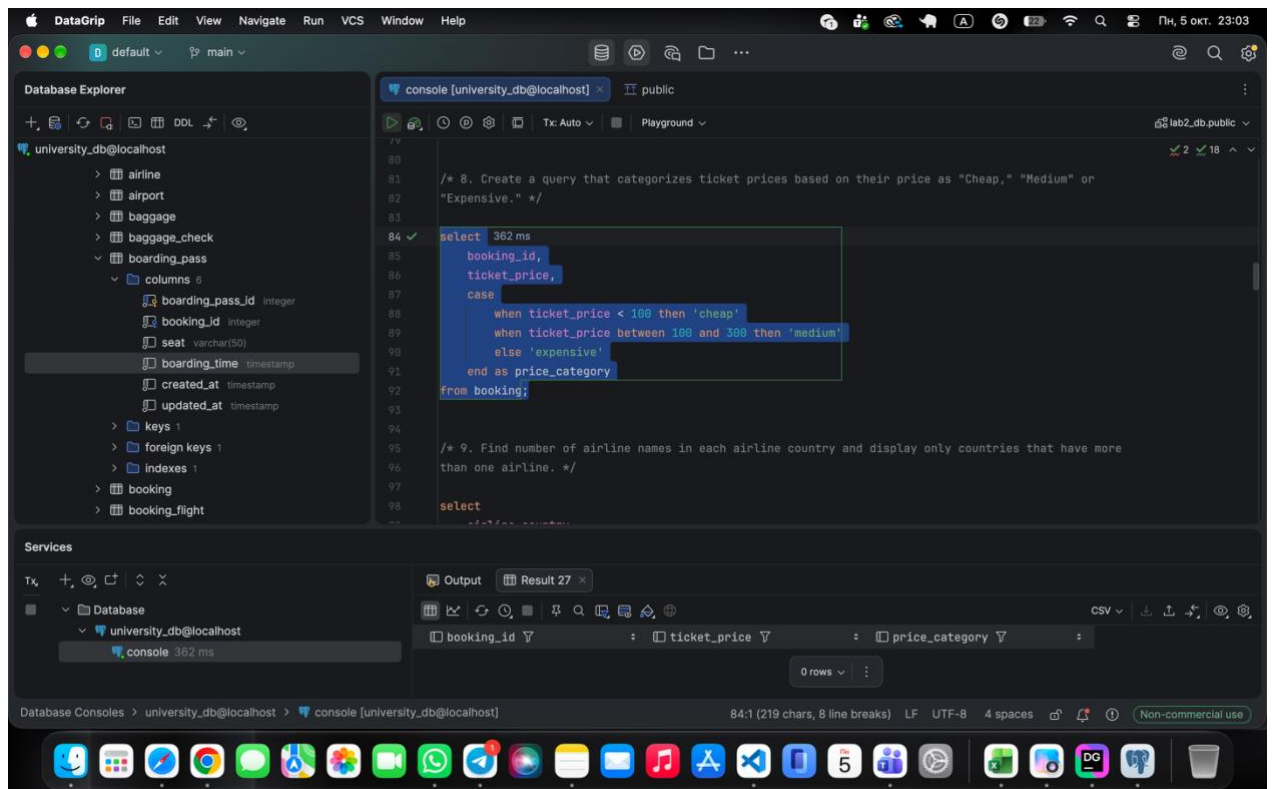
6. Find flight numbers that have been delayed based on the actual arrival time. Display the flight number, scheduled arrival time, actual arrival time, and the delay duration in minutes.



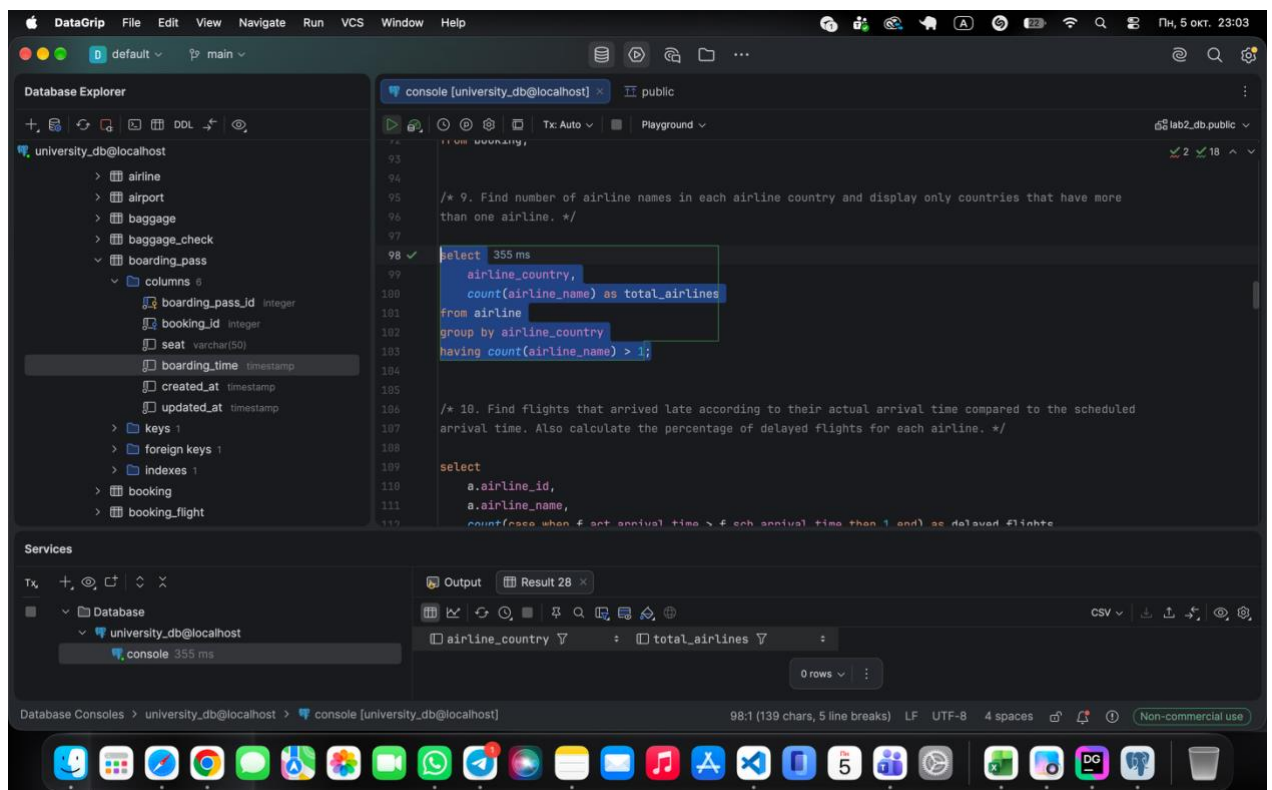
7. Create a query that divides passengers into age groups like 'Young' and 'Adult' based on their birth date. Young passengers age between 18 and 35, Adult passengers age between 36 and 55. Also count how many people we have in each group.



8. Create a query that categorizes ticket prices based on their price as "Cheap," "Medium" or "Expensive."



9. Find number of airline names in each airline country and display only countries that have more than one airline.



10. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time. Also calculate the percentage of delayed flights for each airline.

The screenshot shows the DataGrip interface with a SQL query in the console. The query calculates the percentage of delayed flights for each airline. The results table shows two airlines: AirEasy and FlyHigh, both with 0 delayed flights and 0 percentage.

```
/* 10. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time. Also calculate the percentage of delayed flights for each airline. */  
  
select 390 ms  
  a.airline_id,  
  a.airline_name,  
  count(case when f.act_arrival_time > f.sch_arrival_time then 1 end) as delayed_flights,  
  count(f.flight_id) as total_flights,  
  round(  
    (count(case when f.act_arrival_time > f.sch_arrival_time then 1 end)::decimal / nullif(count(f.flight_id), 0)) *  
    2  
  ) as delayed_percentage  
from airline a  
left join flights f 1<->0.m: on a.airline_id = f.airline_id  
group by a.airline_id, a.airline_name;  
  
/* 11. Find each airline's total number of flights and average flight delay. Display only airlines that have at least 3 flights and an average delay greater than 10 minutes. */
```

airline_id	airline_name	delayed_flights	total_flights	delayed_perce
1002	AirEasy	0	0	0
1003	FlyHigh	0	0	0

11. Find each airline's total number of flights and average flight delay. Display only airlines that have at least 3 flights and an average delay greater than 10 minutes.

The screenshot shows the DataGrip interface with a SQL query in the console. The query calculates the average flight delay for each airline. The results table shows 0 rows, indicating no airlines meet the criteria of having at least 3 flights and an average delay greater than 10 minutes.

```
/* 11. Find each airline's total number of flights and average flight delay. Display only airlines that have at least 3 flights and an average delay greater than 10 minutes. */  
  
select 369 ms  
  a.airline_id,  
  a.airline_name,  
  count(f.flight_id) as total_flights,  
  avg(  
    case  
      when f.act_arrival_time > f.sch_arrival_time  
      then extract(epoch from (f.act_arrival_time - f.sch_arrival_time)) / 60  
      else 0  
    end  
  ) as avg_delay_minutes  
from airline a  
join flights f 1<->1.m: on a.airline_id = f.airline_id  
group by a.airline_id, a.airline_name;
```

airline_id	airline_name	total_flights	avg_delay_minutes
0 rows			

12. Find the top 3 most expensive tickets and display the passenger name, flight number, airline name, and ticket price.

The screenshot shows the DataGrip interface with a SQL console. The query is as follows:

```
148
149
150 /* 12. Find the top 3 most expensive tickets and display the passenger name, flight number, airline
151 name, and ticket price. */
152
153 select 364 ms
154   concat(p.first_name, ' ', p.last_name) as passenger_name,
155   f.flight_id,
156   a.airline_name,
157   b.ticket_price
158 from booking b
159 join passengers p 1.n<>1: on b.passenger_id = p.passenger_id
160 join flights f on b.flight_id = f.flight_id
161 join airline a 1.n<>1: on f.airline_id = a.airline_id
162 order by b.ticket_price desc
163 limit 3;
```

The output shows 0 rows.

13. Calculate the total amount spent by each passenger on flight tickets. Display the passenger's name, number of purchased tickets, total amount spent, and average ticket price.

The screenshot shows the DataGrip interface with a SQL console. The query is as follows:

```
164
165
166 /* 13. Calculate the total amount spent by each passenger on flight tickets. Display the passenger's
167 name, number of purchased tickets, total amount spent, and average ticket price. */
168
169 select 365 ms
170   concat(p.first_name, ' ', p.last_name) as passenger_name,
171   count(b.booking_id) as tickets_purchased,
172   sum(b.ticket_price) as total_amount_spent,
173   round(avg(b.ticket_price), 2) as average_ticket_price
174 from passengers p
175 join booking b 1<->1.n: on p.passenger_id = b.passenger_id
176 group by p.passenger_id, p.first_name, p.last_name;
```

The output shows 0 rows.

14. Analyze airport traffic. For each airport, calculate the number of departing flights, the number of arriving flights, and the total number of flights. Display only airports with more than 10 total flights and sort them by total traffic in descending order.

The screenshot shows the DataGrip application interface. On the left, the 'Database Explorer' pane displays the schema for 'university_db@localhost', including tables like 'airline', 'airport', 'baggage', 'baggage_check', 'boarding_pass', and 'columns'. The 'columns' table is expanded, showing fields like 'boarding_pass_id', 'booking_id', 'seat', 'boarding_time', 'created_at', and 'updated_at'. The main editor displays a SQL query for analyzing airport traffic. The query calculates the number of departing flights, arriving flights, and total flights for each airport, filtering for airports with more than 10 total flights and sorting by total traffic in descending order. The query is as follows:

```
select
  ap.airport_id,
  ap.airport_name,
  count(case when f_dep.departing_airport_id is not null then 1 end) as departing_flights,
  count(case when f_arr.arriving_airport_id is not null then 1 end) as arriving_flights,
  (count(case when f_dep.departing_airport_id is not null then 1 end) +
   count(case when f_arr.arriving_airport_id is not null then 1 end)) as total_flights
from airport ap
left join flights f_dep on ap.airport_id = f_dep.departing_airport_id
left join flights f_arr on ap.airport_id = f_arr.arriving_airport_id
group by ap.airport_id, ap.airport_name
having (count(case when f_dep.departing_airport_id is not null then 1 end) +
       count(case when f_arr.arriving_airport_id is not null then 1 end)) > 10
order by total_flights desc;
```

The 'Output' pane at the bottom shows the results of the query, with columns for 'airport_id', 'airport_name', 'departing_flights', 'arriving_flights', and 'total_flights'. The status bar at the bottom indicates that the query executed successfully, returning 33 rows in 183ms.